Speech Processing Project

Oct 19, 2026 · @mohsen Rashwan

Building a Labelled Speech Corpus from Online Videos

Goal

Students will build a Python application that:

1. Downloads a video from the web by pasting its **link** into the app.
2. Extracts the **audio** track from the video.
3. Generate a UI with spectrogram to help visualize the audio segments in the frequency domain. Then, segments the audio into **speech, music, silence and mixed** regions and labels each one automatically. The labeling should appear on the spectrogram with the ability to adjust the border between any two regions by hand.
4. Cuts the speech and mixed regions into segments of **5 to 15 seconds** at real pauses.
5. Calculates the **SNR** of every segment (clean or noisy).
6. Transcribes every speech segment with **two different ASR systems** and accepts the transcript when they agree. When they differ slightly, the annotator listens and chooses the correct one; otherwise, the segment is labelled **difficult**.
7. Processes **3 different videos**, each at least **one hour** long.
8. A human annotator should be able to **override** any automatic decision.

The result is a labelled speech corpus: every segment is labelled speech, music or mixed, and every speech segment has its SNR and its text transcription.

Teams: 3 students. **Duration:** 5 weeks. **Language:** Python. **Audio:** Arabic videos (different Arabic countries dialect).

Main Steps

Step 1 — Download the Video

- The user pastes a video link into the app and presses **Download**.
- Save the video and its information: link, title, channel, duration and **license**.
- Prefer Creative Commons or public-domain videos.
- You may try: yt-dlp: https://github.com/yt-dlp/yt-dlp, or any alternative.

Step 2 — Extract the Audio

- Extract the audio and convert it to **16 kHz, mono, 16-bit WAV**.

- Tool: ffmpeg: https://ffmpeg.org (call it from Python with `subprocess`).

Step 3 — Segment and Label Speech, Music and Mixed

- Generate a UI with spectrogram to help visualize the audio segments in the frequency domain.
- Split the whole audio into regions and label each one **speech**, **music, silence** or **mixed** (speech over music).
- Give each region a **confidence score** (0 to 1) for the confidence of the automatic decision.
- Use any of these tools, or another one if it works better on your videos:
    - inaSpeechSegmenter: https://github.com/ina-foss/inaSpeechSegmenter.
    - PANNs (AudioSet tags "Speech" and "Music"): https://github.com/qiuqiangkong/panns_inference.

Step 4 — Cut Speech and Mixed Regions into 5–15 s Segments

- Cut only at **real pauses** between words or sentences, never in the middle of a word.
- Keep about 0.1–0.2 s of silence at each edge of a segment.
- Tool: Silero VAD: https://github.com/snakers4/silero-vad (or your own energy-based pause detector).

Step 5 — Calculate the SNR

- Compute the SNR of every segment with two methods, for example:
    - **energy method:** speech frames versus the quietest (pause) frames;
    - **WADA-SNR** (Kim and Stern, 2008).
- Label each segment **clean** or **noisy** (start with a threshold of 20 dB and justify your final choice).

Step 6 — Transcribe with Two ASR Systems

Pass every speech segment, clean or noisy, through two **different** ASR systems like:

- Whisper large-v3 (via faster-whisper): https://github.com/SYSTRAN/faster-whisper
- Meta MMS: https://huggingface.co/facebook/mms-1b-all (or another Arabic model from the Open Universal Arabic ASR Leaderboard on Hugging Face).
- If one or both of the above speech recognition engines is/are not good enough, try other available engines.

Before comparing, **normalize** the Arabic text:

1. remove diacritics and (tatweel كشيدة)
2. Remove Hamzat (إ, أ, ا) to just (ا)

3. Convert all (ة ,ه) to just (ه)
4. Convert all (ى ,ي) to (ي)

and remove punctuation. Then compute WER between the two transcripts with jiwer tool: https://github.com/jitsi/jiwer

Decide the status of each segment:

Case	Example rule (tune it)	Status
Both transcripts agree	identical after normalization, or CER ≤ 2%	**accepted**
Different in a few words	at most 3 different words	the annotator **listens and chooses** A, B or corrects the text → **accepted**
Very different	more than 3 different words	**difficult**

Step 7 — Process Three Long Videos

- Run all steps on **3 different videos**, each at least **one hour** long (e.g. a lecture, a talk show with music, a documentary).
- For every segment save: start, end, label (speech / music / mixed), SNR, clean / noisy, both transcripts, final transcript and status, in a CSV file per video.

Example summary table (the numbers are only an example):

Video	Duration	Speech	Music	Mixed	Segments	Mean SNR	Accepted	Chosen by hand	Difficult
Video 1 — lecture	62 min	54 min	2 min	3 min	410	24 dB	72%	20%	8%
Video 2 — talk show	75 min	51 min	9 min	12 min	455	17 dB	58%	27%	15%
Video 3 — documentary	68 min	40 min	10 min	15 min	380	13 dB	45%	30%	25%

Step 8 — Human Override in the App

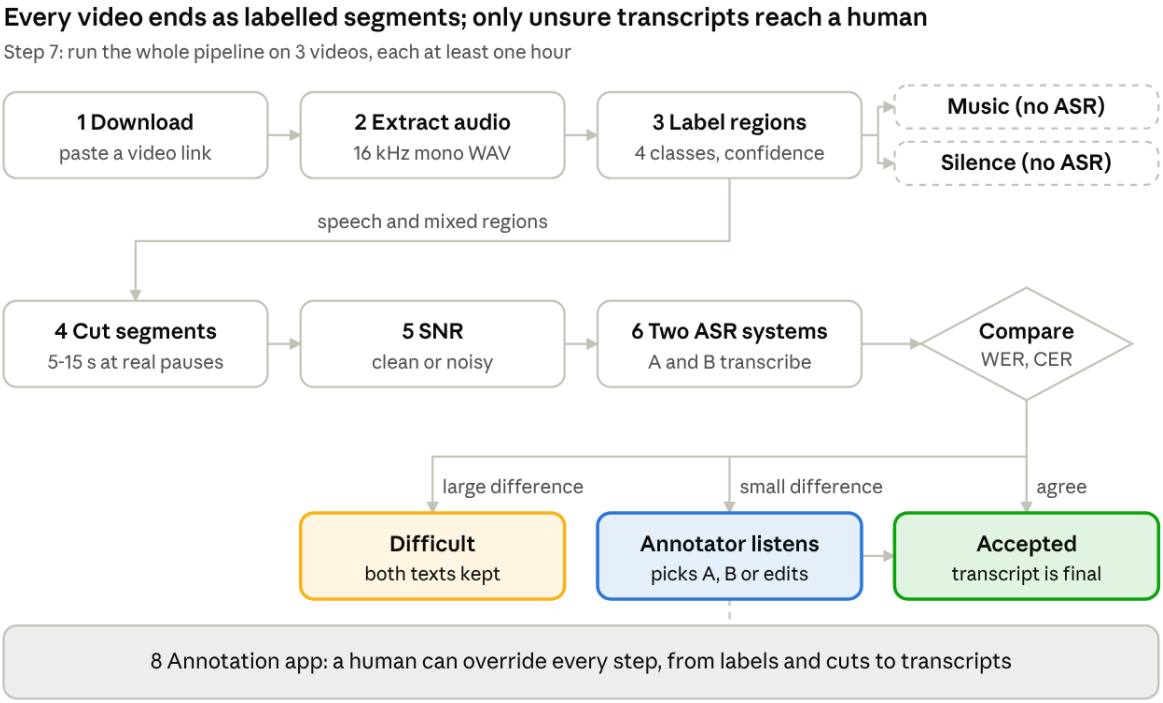
The annotator must be able to:

- **listen** to any segment, and around any cut point;
- **change the label** of any region or segment (speech / music / mixed);

- **move, split, merge, delete or add** segments;
- **choose or edit the transcript** when the two ASR systems differ;
- **change** clean / noisy when the SNR estimate is clearly wrong;
- **save** the work and continue later.

Every human change is recorded (what was changed and by whom). Any Python UI is accepted, e.g. Gradio, Streamlit or PyQt.

The diagram below shows the whole pipeline.



corpus pipeline · 8 tasks, 3 transcript outcomes

Schedule — 5 Weeks, Teams of 3

Suggested roles (every member must still understand and explain the whole pipeline):

- **Student 1 — Audio:** Steps 1–4 (download, audio extraction, speech / music / mixed labelling, segmentation).
- **Student 2 — Quality and ASR:** Steps 5–6 (SNR, the two ASR systems, text-normalization, agreement rule).
- **Student 3 — App:** Step 8 (annotation app) and saving the results to CSV.
- **All three:** Step 7 (the three videos), the human review, the evaluation and the report.

Week	Work	Show at the end of the week
1	Install tools; choose the 3 videos (with license); Steps 1–2; first test of Step 3 on 10 minutes	the 3 approved links; audio of one video; first labels
2	Steps 3–5 on one full video; install both ASR systems; first app (play and relabel)	regions and 5–15 s segments with SNR for video 1
3	Step 6 with the agreement rule; app: choose transcript, move and split segments	CSV of video 1 with transcripts and status
4	Run videos 2 and 3; review all segments that need a human choice; check the results by hand on 10 minutes per video	3 complete CSV files, none left waiting for review
5	Evaluation, report and demo	report, code, live demo

📊 Deliverables

1. **Report** of about 10–12 pages, IEEE style, containing:
   - the three videos (link, duration, content, license);
   - the final labeled segments of audio with the name of each segment .wav with its corresponding text in a .txt files.
   - the method and the parameters of each step, and why you chose them;
   - the results per video (summary table as in Step 7, segment-length histogram, SNR histogram);
   - a check by hand on 10 minutes per video: how many region labels were wrong, how many cuts fell inside a word, how many accepted transcripts had errors;
   - how many automatic decisions the annotator changed, and the review time per hour of audio;
   - at least 5 examples of errors, with their time in the video and the reason.
2. **Python code** (download, audio, segmentation, SNR, ASR, agreement, app) with a README that explains how to install and run it.
3. **Results:** one CSV file per video with all segments, and the segment audio files.
4. **Live demo** (15 minutes): paste a new link, run the pipeline, and correct a few segments in the app.

Grading

Part	Points
Steps 1–2: download and audio	5
Step 3: speech / music / silence / mixed labelling	10
Step 4: 5–15 s segments at real pauses	5
Step 5: SNR	5
Step 6: two ASR systems and the agreement rule	20
Step 7: three one-hour videos fully processed	10
Step 8: annotation app with human override	10
Report and demo	25
Total	**100**

- The total mark will be multiplied with the quality score collected from a test video that has been completely labeled by hand (golden reference).
- Using open-source tools and AI assistants is allowed if you say where you used them in the report. Every student must be able to explain any part of the code during the demo.
- Any added feature will be evaluated, and the team will gain a bonus mark.
- For every added 3 hours/team an extra bonus will be gained, with one condition, the quality is really high and accurate.

📚 Reference Reading

- Whisper: https://arxiv.org/abs/2212.04356
- MMS (Massively Multilingual Speech): https://arxiv.org/abs/2305.13516
- PANNs (audio tagging, speech / music): https://arxiv.org/abs/1912.10211
- Emilia (a large speech dataset built from online video, with a similar pipeline): https://arxiv.org/abs/2407.05361
- WADA-SNR: C. Kim and R. M. Stern, "Robust Signal-to-Noise Ratio Estimation Based on Waveform Amplitude Distribution Analysis", Interspeech 2008.
- MUSAN dataset (speech, music and noise): https://www.openslr.org/17/